

Agenda

- Context
- useEffect
- ~~Redux~~

Use Effect

- used to handle component life cycle events
- accepts 2 parameters
 - 1st: function reference which will be called in different events
 - 2nd: dependency array
- component mounted/launched event
 - this event occurs only once when the component gets loaded/mounted
 - the dependency array must be empty

```
import { useEffect } from 'react'

function App() {
  useEffect(() => {
    console.log('component got loaded')
  }, [])

  return (
    <div>
      <h2>App component</h2>
    </div>
  )
}
```

- component getting unmounted/unloaded
 - this event is fired only once when the component is unmounted/unloaded
 - dependency array must be empty

```
import { useEffect } from 'react'

function App() {
  useEffect(() => {
    console.log('component got loaded')

    return () => {
      console.log('component got unloaded')
    }
  }, [])

  return (
    <div>
      <h2>App component</h2>
    </div>
  )
}
```

```

    </div>
  )
}

```

- a state is changed is updated because of dependency

```

import { useEffect, useState } from 'react'

function App() {
  const [counter, setCounter] = useState(0)

  // when there is change in the counter, the useEffect will be called
  useEffect(() => {
    console.log('updated counter')
  }, [counter])

  return (
    <div>
      <h2>App component</h2>
    </div>
  )
}

```

- component state changed
 - required to perform an action when any of the component state is changed
 - dependency array must not be provided

```

import React, { useEffect, useState } from 'react'

function Counter() {
  const [counter1, setCounter1] = useState(0)
  const [counter2, setCounter2] = useState(0)

  useEffect(() => {
    // this function gets called when there is change in state
    console.log(`state changed`)
  })

  return (
    <div>
      <h1>Counter</h1>
      <div>counter1: {counter1}</div>
      <div>counter2: {counter2}</div>
      <button onClick={() => setCounter1(counter1 + 1)}>
        update counter1
      </button>
      <button onClick={() => setCounter2(counter2 + 1)}>
        update counter2
      </button>
    </div>
  )
}

```

```
    </div>
  )
}
export default Counter
```

useContext()

- used to use context in an application
- context
 - information which is required to shared with multiple components without props drilling
 - It is a solution for props drilling
- props drilling
 - sharing information from one component with multiple (other or child components) components
 - try to avoid it as it will introduce many bugs

```
// Counter.jsx
import { CounterContext } from './App'

function Counter() {
  // read the members from context
  const { counter, setCounter } = useContext(CounterContext)

  return (
    <div>
      <div>counter = {counter}</div>
      <button onClick={() => setCounter(counter + 1)}>update</button>
    </div>
  )
}
```

- NOTE
 - the components with which you want to share the information, must be declared inside the Providers' list
 - if a component is not in the Providers list, the useContext() returns undefined

```
// App.jsx

import { useState, createContext } from 'react'
import Counter from './Counter'

// create context
export const CounterContext = createContext()

function App() {
  const [counter, setCounter] = useState(0)

  return (
    <div>
```

```
        <CounterContext.Provider value={{ counter, setCounter }}>
          <Counter />
        </CounterContext.Provider>
      </div>
    )
  }
```

Q.Session storage vs context

- The session storage will share the data with all components but will not help in render
- Context helps in render automatically
- After refresh the data still remains same in session storage however it resets after refresh if kept in context
- As per the requirement the out of these we should be using.

SUNBEAM INFOTECH